

# DECODELABS INDUSTRIAL TRAINING PROGRAM 2026

FULL STACK PROJECT 3 DATABASE INTEGRATION

SUBMITTED BY LAIBA ZAHID

# 1. EXECUTIVE SUMMARY

This project presents the development of a **Database-Integrated Book Library Management API** using Node.js, Express.js, and MySQL with Prisma ORM. The application enables **permanent data storage** through CRUD (Create, Read, Update, Delete) operations with a MySQL database.

The system incorporates:

- **Database schema design** with proper constraints
- **Full CRUD operations** (POST, GET, PUT, DELETE)
- **Data validation** at application and database level
- **SQL injection protection** via parameterized queries (Prisma)
- **RESTful API design** with proper HTTP status codes
- **Interactive frontend** for user-friendly data management

The project demonstrates **data persistence**, bridging the gap between application logic and permanent storage.

# 2. PROJECT INFORMATION

## 2.1 Project Details

| Field        | Details                                     |
|--------------|---------------------------------------------|
| Project Name | Full Stack Project 3 – Database Integration |
| Domain       | Backend Development / Database Integration  |
| Platform     | Node.js + Express.js + MySQL + Prisma       |

# 3. PROJECT DESCRIPTION

The Book Library Management API is a RESTful backend service developed using Node.js and Express.js, integrated with a MySQL database using Prisma ORM. The API allows users to manage book records permanently through standardized HTTP methods.

**The application performs:**

- Book record creation (CREATE)
- All books retrieval (READ ALL)
- Individual book retrieval (READ ONE)
- Book record updates (UPDATE)
- Book record deletion (DELETE)
- Data validation and constraints
- Permanent data storage in MySQL

## 4. PROJECT OBJECTIVES

1. Design a database schema for book management
2. Connect backend with MySQL database using Prisma ORM
3. Implement full CRUD operations (Create, Read, Update, Delete)
4. Implement server-side validation
5. Use proper HTTP status codes
6. Prevent SQL injection using parameterized queries
7. Return structured JSON responses
8. Build an interactive frontend interface
9. Test all endpoints using Postman
10. Ensure data persistence (survives server restarts)

## 5. KEY FEATURES

| Feature               | Description             |
|-----------------------|-------------------------|
| GET /api/books        | Fetch all book records  |
| GET /api/books/:id    | Fetch single book by ID |
| POST /api/books       | Add a new book          |
| PUT /api/books/:id    | Update an existing book |
| DELETE /api/books/:id | Delete a book record    |
| Database              | MySQL with Prisma ORM   |
| Data Constraints      | UNIQUE, NOT NULL, CHECK |

| Feature                  | Description                      |
|--------------------------|----------------------------------|
| Frontend                 | Interactive HTML/CSS interface   |
| SQL Injection Protection | Parameterized queries via Prisma |

## 6. REQUIREMENTS CHECKLIST

### 6.1 Requirement Verification

| #  | Requirement               | Status      | Evidence                           |
|----|---------------------------|-------------|------------------------------------|
| 1  | Database Schema Design    | ✓ Completed | Prisma Schema                      |
| 2  | MySQL Database Connection | ✓ Completed | .env configuration                 |
| 3  | CREATE Operation (POST)   | ✓ Completed | /api/books endpoint                |
| 4  | READ Operation (GET All)  | ✓ Completed | /api/books endpoint                |
| 5  | READ Operation (GET One)  | ✓ Completed | /api/books/:id endpoint            |
| 6  | UPDATE Operation (PUT)    | ✓ Completed | /api/books/:id endpoint            |
| 7  | DELETE Operation (DELETE) | ✓ Completed | /api/books/:id endpoint            |
| 8  | Data Validation           | ✓ Completed | Server-side + Database constraints |
| 9  | SQL Injection Protection  | ✓ Completed | Parameterized queries (Prisma)     |
| 10 | Proper HTTP Status Codes  | ✓ Completed | 200, 201, 400, 404, 409, 500       |
| 11 | JSON Responses            | ✓ Completed | Structured API responses           |
| 12 | Frontend Interface        | ✓ Completed | HTML/CSS/JavaScript                |

| #  | Requirement    | Status      | Evidence                 |
|----|----------------|-------------|--------------------------|
| 13 | API Testing    | ✓ Completed | Postman / Thunder Client |
| 14 | Error Handling | ✓ Completed | try-catch blocks         |

## 6.2 Completion Metrics

| Metric                 | Value |
|------------------------|-------|
| Total Requirements     | 14    |
| Completed Requirements | 14    |
| Completion Percentage  | 100%  |

# 7. TECHNOLOGY STACK

## 7.1 Backend Technologies

| Technology | Purpose                         |
|------------|---------------------------------|
| Node.js    | JavaScript Runtime Environment  |
| Express.js | Web Framework                   |
| MySQL      | Relational Database             |
| Prisma ORM | Database ORM and Migration Tool |
| JavaScript | Programming Language            |

## 7.2 Tools and Services

| Tool                     | Purpose                 |
|--------------------------|-------------------------|
| XAMPP                    | MySQL Database Server   |
| phpMyAdmin               | Database Management     |
| Postman / Thunder Client | API Testing             |
| Visual Studio Code       | Development Environment |
| GitHub                   | Version Control         |
| Command Prompt           | Server Management       |

## 8. DATABASE SCHEMA DESIGN

### 8.1 Books Table Schema

```
CREATE TABLE books (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  title VARCHAR(255) NOT NULL,  
  author VARCHAR(255) NOT NULL,  
  year INT NOT NULL,  
  genre VARCHAR(100) NOT NULL,  
  createdAt DATETIME DEFAULT CURRENT_TIMESTAMP,  
  updatedAt DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  UNIQUE KEY unique_book (title, author)  
);
```

### 8.2 Database Constraints

| Constraint  | Field | Purpose                         |
|-------------|-------|---------------------------------|
| PRIMARY KEY | id    | Unique identifier for each book |

| Constraint | Field                      | Purpose                            |
|------------|----------------------------|------------------------------------|
| NOT NULL   | title, author, year, genre | Ensures required fields are filled |
| UNIQUE     | (title, author)            | Prevents duplicate book entries    |
| DEFAULT    | createdAt, updatedAt       | Auto-timestamps                    |

## 9. API ENDPOINTS DOCUMENTATION

### 9.1 Endpoint Overview

| Method | Endpoint       | Description               | Status Codes       |
|--------|----------------|---------------------------|--------------------|
| GET    | /api/health    | Database connection check | 200, 500           |
| GET    | /api/books     | Get all books             | 200, 500           |
| GET    | /api/books/:id | Get single book           | 200, 404, 500      |
| POST   | /api/books     | Add new book              | 201, 400, 409, 500 |
| PUT    | /api/books/:id | Update book               | 200, 404, 400, 500 |
| DELETE | /api/books/:id | Delete book               | 200, 404, 500      |

## 10. DATA VALIDATION RULES

| Field  | Validation Rule        | Error Message                                |
|--------|------------------------|----------------------------------------------|
| Title  | Required and Non-empty | "Please provide: title, author, year, genre" |
| Author | Required and Non-empty | "Please provide: title, author, year, genre" |

| Field     | Validation Rule              | Error Message                                |
|-----------|------------------------------|----------------------------------------------|
| Year      | Required, Numeric, 1000-2026 | "Year must be between 1000 and 2026"         |
| Genre     | Required and Non-empty       | "Please provide: title, author, year, genre" |
| Duplicate | (title, author) unique       | "This book already exists!"                  |

## 11. HTTP STATUS CODES

| Code | Meaning               | Usage                                |
|------|-----------------------|--------------------------------------|
| 200  | OK                    | Successful GET, PUT, DELETE requests |
| 201  | Created               | Record created successfully (POST)   |
| 400  | Bad Request           | Validation failure                   |
| 404  | Not Found             | Book ID not found                    |
| 409  | Conflict              | Duplicate entry                      |
| 500  | Internal Server Error | Unexpected server error              |

## 12. PROJECT FILE STRUCTURE

```

book-library/
├── node_modules/
├── prisma/
│   └── schema.prisma
├── public/
│   └── index.html
├── .env
└── package.json

```



└─ package-lock.json  
└─ server.js

## 13. CHALLENGES AND SOLUTIONS

| Challenge         | Problem                     | Solution                                    | Time    |
|-------------------|-----------------------------|---------------------------------------------|---------|
| Prisma Version 7  | URL in schema not supported | Downgraded to Prisma 5                      | 20 mins |
| MySQL Connection  | Connection refused          | Started XAMPP MySQL                         | 5 mins  |
| Database Creation | Database not found          | Created via phpMyAdmin                      | 5 mins  |
| Prisma Client     | Client not generated        | Ran <code>npx prisma generate</code>        | 5 mins  |
| Frontend 404      | Static files not serving    | Added <code>express.static('public')</code> | 5 mins  |
| Duplicate Books   | Same book adding twice      | Added UNIQUE constraint                     | 10 mins |
| Year Validation   | Invalid years accepted      | Added CHECK constraint                      | 10 mins |

## 14. API TESTING REPORT

### 14.1 Endpoint Testing

| Endpoint       | Method | Status | Response           |
|----------------|--------|--------|--------------------|
| /api/health    | GET    | Pass   | Database connected |
| /api/books     | GET    | Pass   | All books returned |
| /api/books/1   | GET    | Pass   | Single book        |
| /api/books/999 | GET    | Pass   | 404 Not Found      |

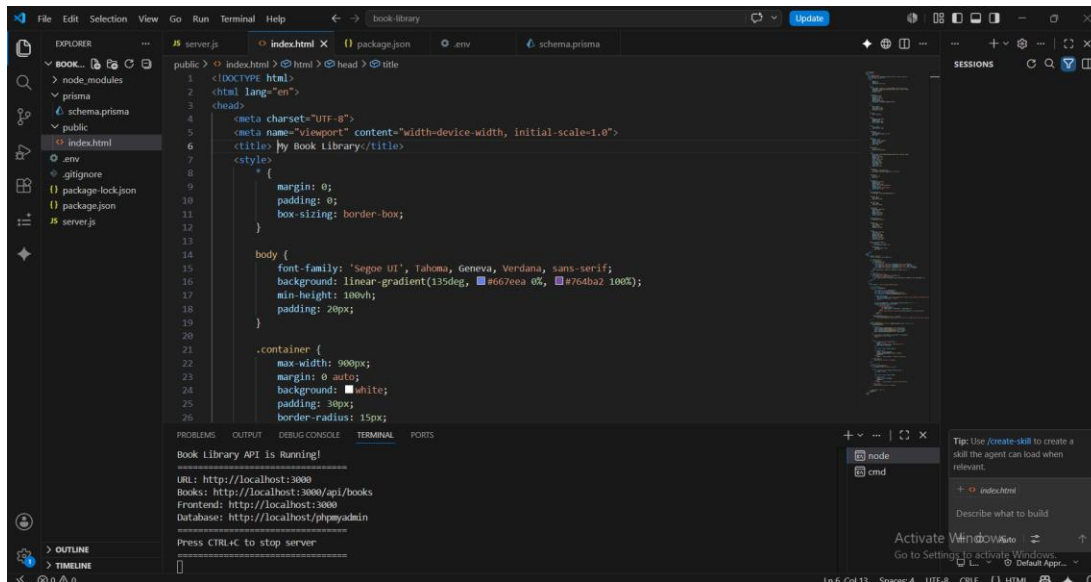
| Endpoint     | Method | Status | Response       |
|--------------|--------|--------|----------------|
| /api/books   | POST   | Pass   | 201 Created    |
| /api/books   | POST   | Pass   | 400 Validation |
| /api/books/1 | PUT    | Pass   | Book updated   |
| /api/books/1 | DELETE | Pass   | Book deleted   |

## 14.2 Validation Testing

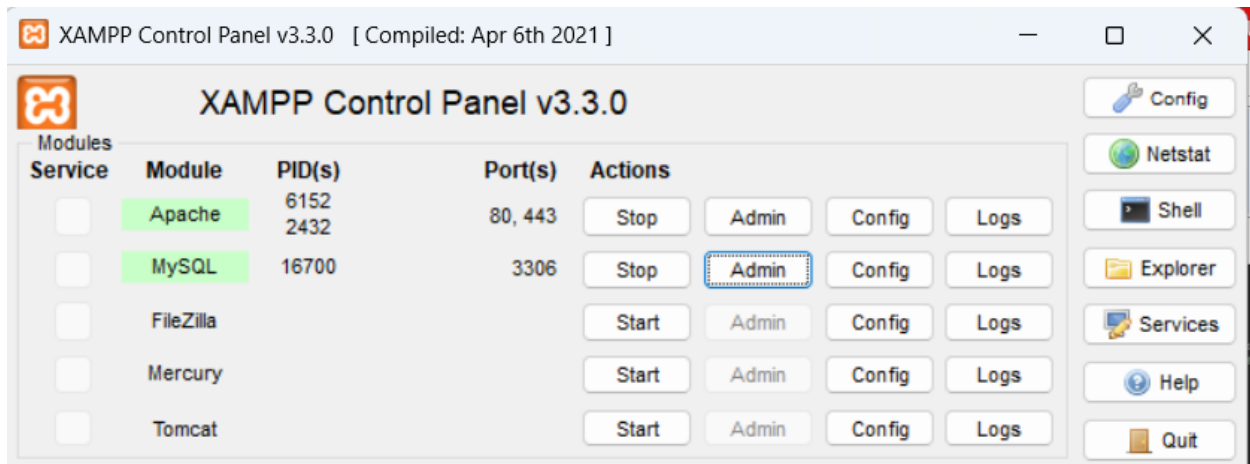
| Test Case      | Input              | Expected    | Status |
|----------------|--------------------|-------------|--------|
| Missing Title  | No title field     | 400 Error   | Pass   |
| Missing Author | No author field    | 400 Error   | Pass   |
| Invalid Year   | year: 500          | 400 Error   | Pass   |
| Empty Fields   | title: ""          | 400 Error   | Pass   |
| Duplicate Book | Same title+author  | 409 Error   | Pass   |
| Valid Data     | All fields correct | 201 Created | Pass   |

## 15. SCREENSHOTS DOCUMENTATION

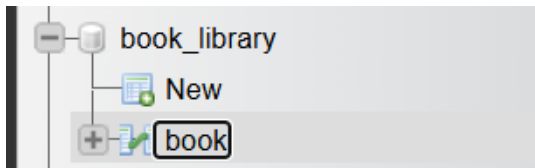
### Screenshot 1: VS Code - Project Structure



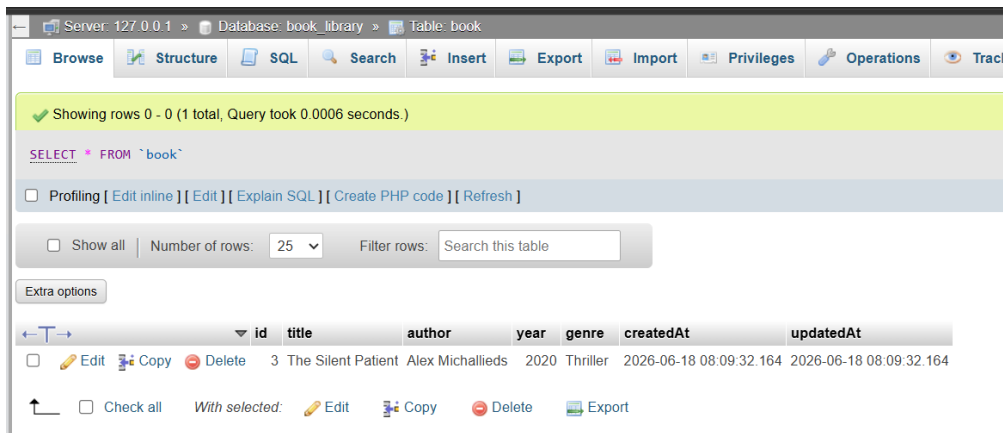
### Screenshot 2: XAMPP - MySQL Running



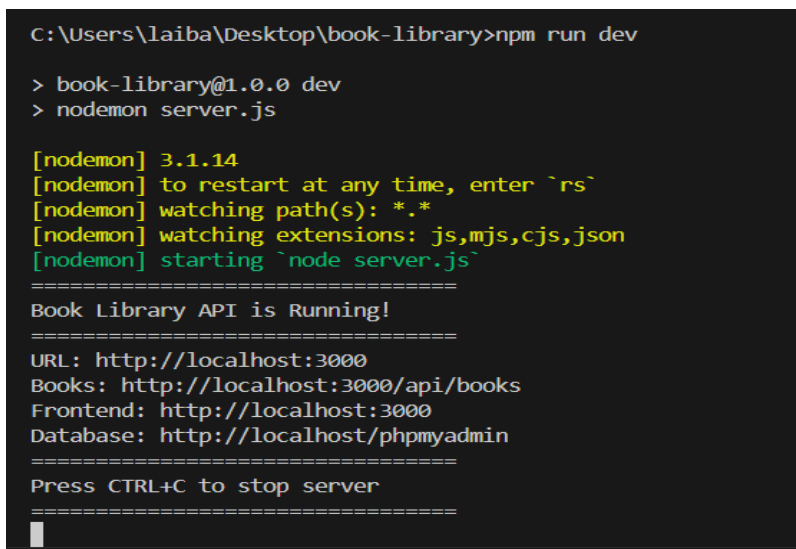
### Screenshot 3: phpMyAdmin - Database and Table



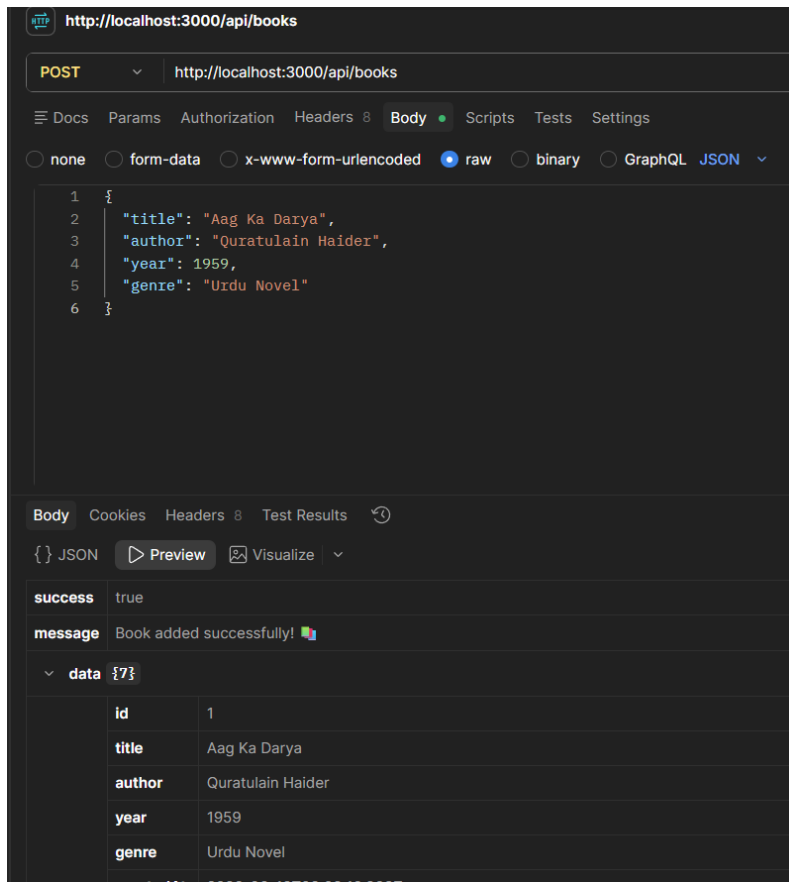
### Screenshot 4: phpMyAdmin - Books Table Data



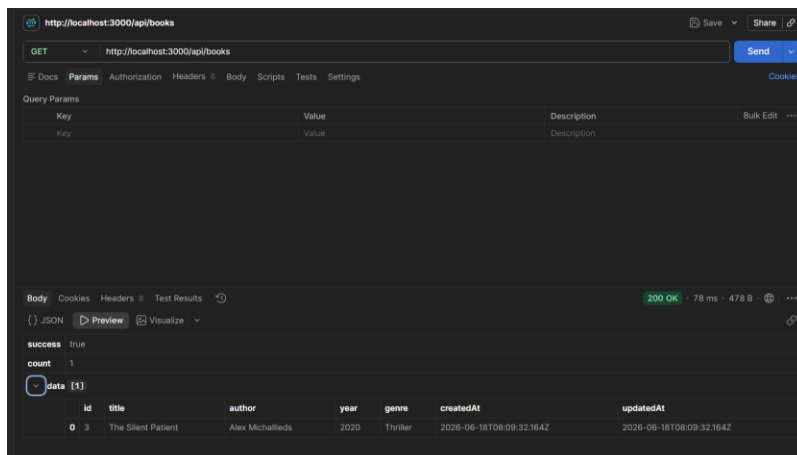
### Screenshot 5: Terminal - Server Running



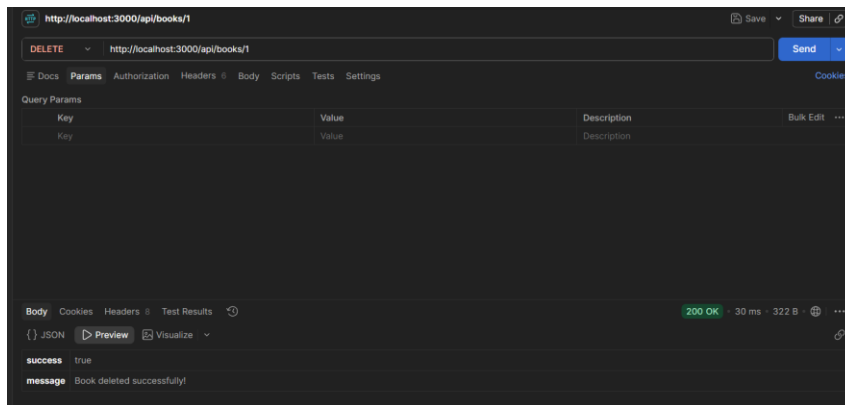
## Screenshot 6: Postman - POST Request (Add Book)



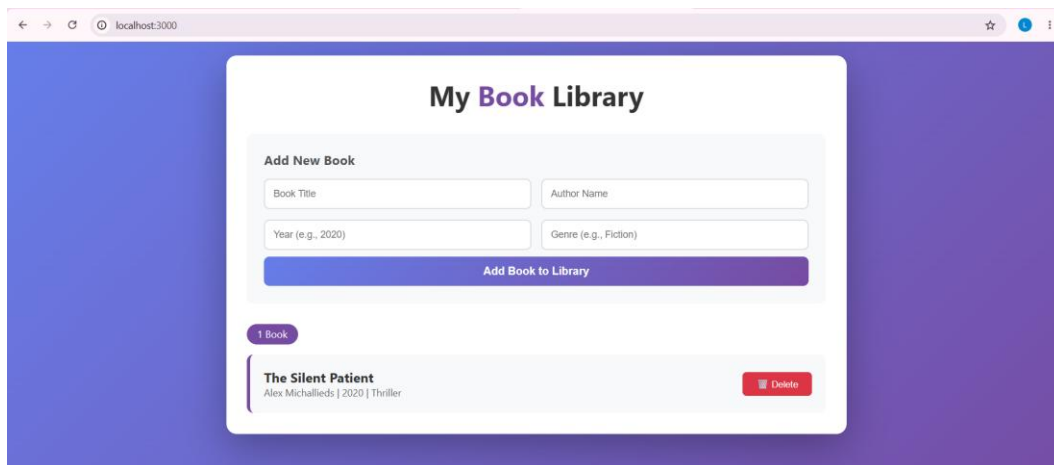
## Screenshot 7: Postman - GET Request (All Books)



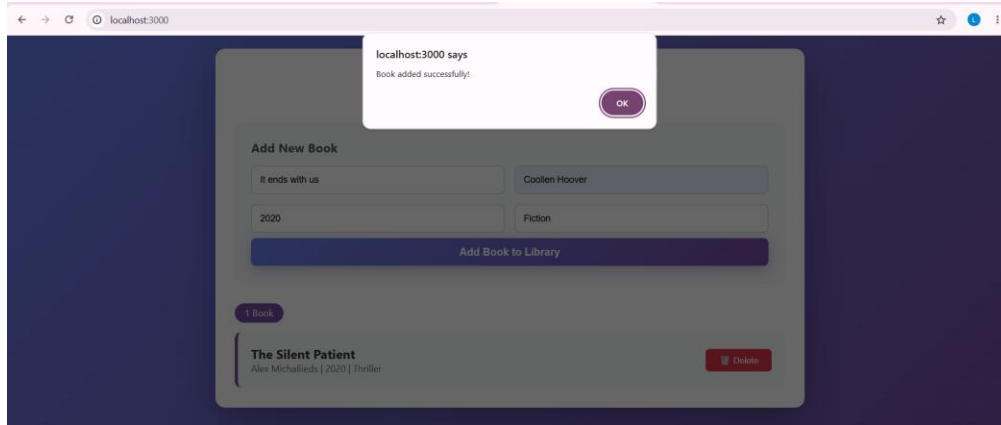
## Screenshot 8: Postman - DELETE Request



## Screenshot 9: Browser - Frontend Homepage



## Screenshot 10: Browser - Adding Book via Frontend



## 16. CONCLUSION AND LEARNINGS

The Book Library Management API was successfully developed and tested according to all project requirements. The project demonstrates proficiency in **database integration**, **CRUD operations**, and **backend development** using Node.js, Express.js, and MySQL with Prisma ORM.

### Key Learnings

1. Database schema design with proper constraints
2. MySQL database connection and management
3. CRUD operations (Create, Read, Update, Delete)
4. Prisma ORM for database interaction
5. SQL injection prevention via parameterized queries
6. Data validation at application and database level
7. RESTful API design and implementation
8. HTTP status codes and error handling
9. Frontend-backend integration
10. API testing using Postman
11. Data persistence (survives server restarts)
12. XAMPP and phpMyAdmin for database management

### Future Improvements

- User authentication (Login/Register)
- Advanced search functionality (by title, author, genre)
- Pagination for large datasets
- Cloud deployment (Render, Railway)
- PostgreSQL database support
- Mobile application development
- Book borrowing/return system

- Reviews and ratings for books

## 17. GITHUB REPOSITORY

Repository Link: <https://github.com/lybah-gif/Decodeslab-project-3>

## 18. DECLARATION

I hereby declare that this project, "Full Stack Project 3 – Database Integration," is my original work. All requirements specified in the Decodelabs Industrial Training Program project brief have been successfully completed.

**The project was developed using:**

- Node.js
- Express.js
- MySQL (via XAMPP)
- Prisma ORM
- JavaScript (Vanilla)
- HTML/CSS (Frontend)

The implementation follows industry-standard coding practices and internship guidelines.

## STUDENT INFORMATION

Name Laiba Zahid

Batch 2026

Project Full Stack Project 3

Date June 18, 2026

---